

Technical Writeup

Vulnerable Browser Extension

CS 4277 Final Project Team Members: Maret Rudin-Aulenbach, Jimmy Baek, Juan Guerrero

1. Project Overview

This project is a Chrome browser extension, built on Manifest V3, that summarizes webpage content for the user. The extension reads text from the currently active tab, generates a short summary, and stores a history of past summaries. The user can review their history through the extension popup or a dedicated options page.

For our project, we intentionally implemented two security vulnerabilities into a “vulnerable” build of the extension, then developed a hardened “secure” build that addresses each vulnerability with targeted defenses. A set of demo pages (`malicious_demo/`) allows a user to simulate both attacks and defenses.

Extension Architecture

The extension consists of three main components:

Component	File	Role
Popup UI	<code>popup.html / popup.js</code>	User facing toolbar popup with “Summarize” and “View History” buttons.
Background Service Worker	<code>background.js</code>	Receives text from the content script or popup, produces a summary, and manages persistent history storage via <code>chrome.storage.local</code> .
Content Script	<code>content.js</code>	Injected into every page (<code><all_urls></code>). Handles page level text collection and cross context messaging.
Options Page		

Component	File	Role
	options.html / options.js	Displays stored summary history inside the extension's own UI context.

The two builds, vulnerable/ and secure/, share this architecture but differ in how they collect data and who they communicate with.

2. Attack 1: Message-Based History Theft

2.1 Vulnerability

The vulnerable extension's content script (vulnerable/content.js) registers a `window.addEventListener("message", ...)` handler that listens for incoming `postMessage` events. Critically, this handler performs no origin validation and processes messages from any source, including arbitrary web pages the user visits:

```

window.addEventListener("message", async (event) => {
  // Attack 1: intentionally accepts messages from any origin.
  if (event.data?.type === "GET_HISTORY") {
    chrome.runtime.sendMessage({ type: "GET_HISTORY_FOR_POPUP" },
      (response) => {
        window.postMessage(
          {
            type: "HISTORY_RESPONSE",
            history: response?.history || [],
            ok: Boolean(response?.ok)
          },
          "*"
        );
      }
    );
  }
});

if (event.data?.type === "SUMMARIZE_PAGE") {
  const allText = collectEntireDomText();
  const response = await requestSummaryFromBackground(allText);
  window.postMessage({ type: "SUMMARY_RESPONSE", response },
    "*");
}
});

```

The `window.postMessage` API allows scripts on the same page to send messages to each other, even across origins. Since the content script can communicate with the website's JavaScript, a malicious page can call `window.postMessage({ type: "GET_HISTORY" }, "*")` and the content script will reply with the user's entire summary history. The background service worker (vulnerable/background.js)

also lacks any sender validation and responds to `GET_HISTORY_FOR_POPUP` from any caller without checking whether the request originated from the extension popup or an untrusted content script.

Furthermore, the response is posted back via `window.postMessage(..., "*")`, broadcasting the history to every listener on the page, including the attacker's script.

2.2 Attack Scenario

1. The user browses the web normally and uses the vulnerable extension to summarize several pages. Summaries accumulate in `chrome.storage.local`.
2. The user visits a malicious website (demonstrated by `malicious_demo/index.html`, disguised as a news article).
3. The malicious page runs a single line of JavaScript:

```
window.postMessage({ type: "GET_HISTORY" }, "*");
```

4. The content script, already injected on the page, receives this message, forwards it to the background worker, and posts the entire history back to the page.
5. The attacker's script captures the response and can record it to an external server for malicious purposes.

2.3 Defense

The secure extension eliminates this attack surface through two changes:

1. We removed the `postMessage` bridge entirely. The secure content script (`secure/content.js`) still has a `window.addEventListener("message", ...)` handler, but it is defended, filtering by origin and processing nothing. If the extension had more features that should be available to any page, those would be handled here, but our summarization and get history features should only be accessible to the user via the extension UI.

```
window.addEventListener("message", (event) => {  
  if (event.origin !== window.location.origin) return;  
  if (!event.data || typeof event.data !== "object") return;  
  // The secure version intentionally does not expose history or  
  // summary APIs here.  
});
```

No page level script can trigger a history lookup or summary via `postMessage` because the handler never forwards those requests.

1. The secure background script (`secure/background.js`) now checks `sender.tab` when processing `GET_HISTORY_FOR_POPUP` messages:

```
// Only extension UI should access history.  
if (message?.type === "GET_HISTORY_FOR_POPUP" && !sender.tab) {  
  getHistory()  
    .then((history) => sendResponse({ ok: true, history })))
```

```

    .catch((err) => sendResponse({ ok: false, error:
      String(err) }));
  return true;
}

```

In Chrome's messaging model, requests from content scripts carry a `sender.tab` object, while requests from the popup or options page do not. By requiring `!sender.tab`, the background worker ensures that only the extension's own privileged UI can retrieve history. As a result, users can only access their summarization history via the authorized extension popup and options pages.

3. Attack 2: Over-Collection of Sensitive DOM Data

3.1 Vulnerability

The vulnerable extension's content script and popup both use the function `collectEntireDomText()` to extract text for summarization. This function performs two excessively broad data collection operations:

```

function collectEntireDomText() {
  const visible = document.body?.innerText || "";
  const hiddenValues = Array.from(
    document.querySelectorAll("input, textarea")
  ).map((el) => `${el.name || el.id || "field"}=${el.value || ""}`
  );

  return [visible, hiddenValues.join("\n")].join("\n").trim();
}

```

`document.body.innerText` captures all rendered text on the page, which on a banking site would include account numbers, routing numbers, balances, and SSNs, even those that appear visually masked with CSS (e.x. `filter: blur(4px)`), because `innerText` reads the computed text content rather than its visual appearance.

`document.querySelectorAll("input, textarea")` enumerates every form element on the page. This includes: - `<input type="password">` fields containing plaintext passwords (the `.value` property always returns the cleartext regardless of the input type). - `<input type="hidden">` fields containing CSRF tokens, session IDs, internal API keys, and other secrets that are never displayed to the user.

Because the summarization pipeline passes this collected text directly to the background worker, which stores it in history, all of this sensitive data ends up in the extension's storage. Combined with Attack 1, an attacker on a separate site could then steal this stored sensitive data remotely.

3.2 Attack Scenario

The demo page `malicious_demo/attack2.html` is a mock bank portal that contains sensitive data in multiple DOM locations:

Data	DOM Location	Visual Appearance
Account numbers (4821-0039-7713)	Visible <div> text	Visible on screen
Routing number (091000019)	Visible <div> text	Visible on screen
Account balances (\$24,817.43)	Visible <div> text	Visible on screen
SSN (123-45-6789)	<input type="hidden">	Not rendered
User password (MyS3cretBanking!)	<input type="password">	Masked with dots
Security answer (Fitzgerald)	<input type="password">	Masked with dots
CSRF token	<input type="hidden">	Not rendered
Session ID	<input type="hidden">	Not rendered
Account token	<input type="hidden">	Not rendered
Email address	Visible <div> text	Visible on screen

When the vulnerable extension’s “Summarize Entire Page” button is clicked while viewing this page, `collectEntireDomText()` captures all of this data, including the password field values, hidden tokens, and SSN, and stores it in the summary history. The demo page’s “Run Attack 2” button simulates this same collection to visually demonstrate what would be captured, with sensitive values highlighted in red in a terminal style overlay.

3.3 Defense

The secure extension addresses this vulnerability through two defense layers:

1. Instead of scraping the entire DOM, the secure extension only summarizes text that the user has explicitly highlighted/selected. The content script (`secure/content.js`) uses `window.getSelection()` instead of `document.body.innerText`:

```
function getUserSelectedText() {
  // Try live selection first; fall back to cached.
  const live = window.getSelection()?.toString().trim() || "";
  return live || cachedSelection;
}
```

The popup button becomes “Summarize Selection” instead of “Summarize Entire Page”, and the popup UI includes a warning banner:

Highlight only non-sensitive text before summarizing. Do not select passwords, IDs, account numbers, or tokens.

If no text is selected, the extension refuses to proceed and displays an error message. This design shifts the control to the user: they choose exactly what goes into the summary, eliminating the automatic over-collection of passwords and hidden fields.

A selection caching mechanism was also implemented to deal with a Chrome specific UX challenge: when the user clicks the extension popup, Chrome shifts focus away from the page, which can clear the active selection. The content script listens for mouseup and selectionchange events to continuously cache the most recent nonempty selection, ensuring it persists through the focus shift.

1. We implemented regex-based sensitive data filtering. Even with selection only input, a user might accidentally select text that contains sensitive values, or a page might display sensitive data inline or hidden where it could be selected alongside safe text. As an additional defense, the secure background worker (secure/background.js) runs a sanitizeSensitiveData() function on all incoming text before generating a summary:

```
const SENSITIVE_PATTERNS = [
  // JWT tokens (three base64url segments)
  { re: /eyJ[A-Za-z0-9_-]+\.[A-Za-z0-9_-]+\.[A-Za-z0-9_-]+/g,
    label: "[JWT_REDACTED]" },
  // SSN ###-##-####
  { re: /\b\d{3}-\d{2}-\d{4}\b/g, label: "[SSN_REDACTED]" },
  // Credit / debit card (4 groups of 4 digits, optional dashes/
  // spaces)
  { re: /\b(?:\d{4}[\s-])?{3}\d{4}\b/g, label:
    "[CARD_REDACTED]" },
  // Routing numbers (9-digit, standalone)
  { re: /\b0[2-9]\d{7}\b/g, label: "[ROUTING_REDACTED]" },
  // CSRF, session, auth, pin, otp, secret key=value pairs
  { re: /(?:csrf|session|token|auth|api[_]?key|pin|otp|secret|
    wire_tok)[=:\s]+[\w.-]{6,}/gi, label:
    "[TOKEN_REDACTED]" },
  // Password key=value pairs
  { re: /(?:password|passwd|pwd)[=:\s]+\S+/gi, label:
    "[PASSWORD_REDACTED]" },
  // 32+ hex character secrets (SHA-like IDs, internal tokens)
  { re: /\b[0-9a-f]{32,}\b/gi, label: "[HEX_SECRET_REDACTED]" },
  // Email addresses
  { re: /[a-zA-Z0-9._%+\-]+@[a-zA-Z0-9.\-]+\.[a-zA-Z]{2,}/g,
    label: "[EMAIL_REDACTED]" }
];
```

This sanitizer catches common sensitive data patterns, including SSNs, credit card numbers, JWT tokens, key value pairs for passwords/CSRF/session tokens, lengthy hex secrets, routing numbers, and email addresses, and replaces them with descriptive [*_REDACTED] labels before the summary is stored. This adds a second layer of protection, however regex filtering may not catch every possible form of sensitive data and thus the user selected input defense is important as well.

6. How to Run the Demos

Please use Google Chrome to run the demos.

Loading an Extension

1. Navigate to `chrome://extensions`.
2. Toggle Developer Mode on.
3. Click “Load unpacked” and select either the `vulnerable/` or `secure/` directory.
4. Use one extension at a time.

Demonstrating Attack 1

1. Load the vulnerable extension.
2. Visit some pages and click “Summarize Entire Page” to create a summarization history.
3. Open `malicious_demo/index.html` in Chrome.
4. Click the “Run Attack 1: Steal Extension History” button.
5. Observe the stolen history displayed on the webpage.

Demonstrating Attack 2

1. Load the vulnerable extension.
2. Open `malicious_demo/attack2.html` in Chrome (the mock bank portal).
3. Click “Run Attack 2: Over-Collect Page Content” on the demo page.
4. Observe the UI showing all captured data. Passwords, tokens, and SSNs are highlighted in red.
5. Additionally, click “Summarize Entire Page” in the extension popup and then “View History” to see the sensitive data stored in the extension’s history.

Demonstrating the Defenses

1. Unload the vulnerable extension and load the secure extension.
2. Attempt attack 1: open `malicious_demo/index.html` and click the attack button. No response is received.
3. Attempt attack 2: navigate to `malicious_demo/attack2.html` and click “Summarize Selection” without selecting text. The extension will not summarize any page content because no text is selected. Then select only safe text (such as a transaction description), summarize, and verify that only the selected text appears in the summary. To test the regex defense, select text that includes sensitive data and confirm that values like SSNs and passwords are redacted in the stored summary.

7. Team Contributions

Team Member	Contributions
Juan Guerrero	

Team Member	Contributions
Maret Rudin-Aulenbach	Built the foundational extension architecture and boilerplate: Manifest V3 configuration, popup UI, background service worker, content script scaffolding, options page, and the storage layer. Developed the vulnerable version of the extension that serves as the baseline for both attacks.
	Designed vulnerabilities and defenses for attacks 1 and 2, as well as implemented the Attack 1 defense, by removing the vulnerable <code>postMessage</code> bridge, adding origin validation, restricting history access to authorized extension UI only (popup and options page), and implementing sender validation (<code>!sender.tab</code>) in the background worker.
	Designed and implemented attack 2 (over-collection of sensitive DOM data). Created the attack 2 demo page.
Jimmy Back	Developed the attack 2 defense: replacing full DOM collection with selection only input, adding the regex based sensitive data filter in the secure extension version.

8. Conclusion

This project demonstrates two vulnerabilities in a browser extension: an unvalidated `postMessage` bridge that exposes summarization history to any webpage, and excessive DOM scraping that captures passwords, hidden tokens, and other sensitive data. The secure version defends against these vulnerabilities by eliminating the `postMessage` vulnerability, restricting history access to the extension UI via `sender.tab` checks, limiting data collection to user selected text, and applying regex filtering to redact sensitive patterns before storage.

Disclosure: Our team used an LLM to assist with aspects of the code and report. LLM-assisted material was reviewed, edited, and understood by the team before submission.